ML & DL : Experiment - 9

Aim:

Implement Recurrent Neural Network (RNN) / LSTM for time series or text data.

Dataset Description:

The IMDB Movie Review Dataset used in this experiment is a built-in dataset provided by the Keras deep learning library. It is a widely used benchmark dataset for binary sentiment classification tasks in Natural Language Processing (NLP). The dataset contains preprocessed movie reviews labeled as either positive or negative. The reviews are already tokenized and converted into sequences of integer word indices based on word frequency. This allows direct usage in deep learning models without manual text cleaning or tokenization. Since each review is paired with a sentiment label (0 or 1), it is a supervised learning dataset suitable for training Recurrent Neural Networks (RNNs) and LSTM models. Due to its balanced class distribution, standardized preprocessing, and moderate dataset size, the IMDB dataset is commonly used in academic research, deep learning tutorials, and hyperparameter tuning experiments for text classification.

- File Type: Built-in Keras dataset (NumPy arrays loaded via TensorFlow/Keras)
- Dataset Size: 50,000 movie reviews (25,000 training samples & 25,000 testing samples)
- Data Format: Integer-encoded sequences representing word indices
- Number of Classes: 2 Classes (0 → Negative Review & 1 → Positive Review)

Column Name	Data Type	Description
Review (Sequence)	Numerical (Integer sequence)	Tokenized movie review is represented as a sequence of word indices based on frequency ranking.
Label	Numerical (Binary: 0 or 1)	Sentiment category corresponding to each review (0 = Negative, 1 = Positive).

Dataset Source: https://keras.io/api/datasets/imdb/

Recurrent Neural Network (RNN) / LSTM

Theory:

Recurrent Neural Networks (RNNs) are a class of deep learning models designed for processing sequential data such as text, speech, and time-series signals. Unlike traditional feedforward neural networks, RNNs maintain a hidden state that captures information from previous time steps, allowing them to model temporal dependencies. However, standard RNNs suffer from the vanishing gradient problem, which makes learning long-term dependencies difficult. To overcome this limitation, Long Short-Term Memory (LSTM) networks were introduced. LSTMs are a specialized type of RNN that use gating mechanisms to control the flow of information, enabling them to retain relevant information over long sequences.

$$X \in \mathbb{R}^{T \times d}$$

where:

- T = length of the sequence (number of time steps / words)
- d = dimension of each input vector (e.g., embedding dimension)

At each time step t, the RNN updates its hidden state as:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

where:

- x_t = input at time step t
- h_t = hidden state at time step t
- W_x = input weight matrix b = bias term
- W_h = recurrent weight matrix

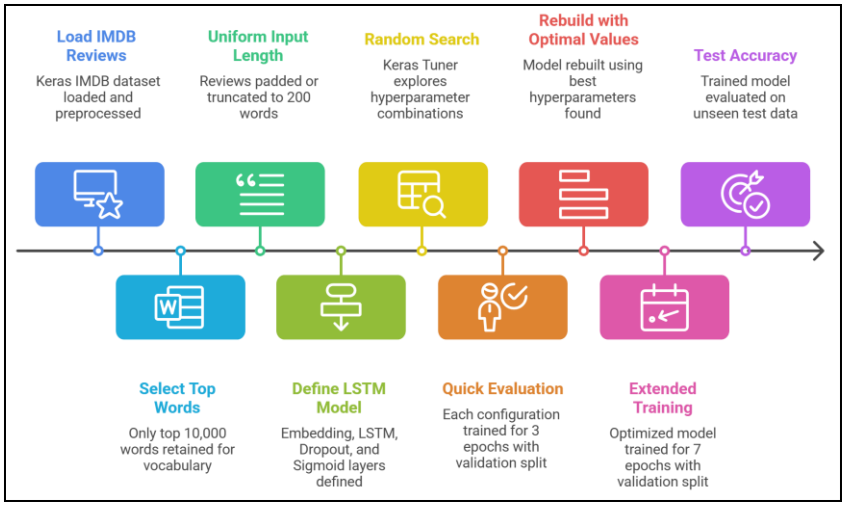
Limitations:

1. **Vanishing and Exploding Gradient Problem:** Traditional Recurrent Neural Networks suffer from the vanishing and exploding gradient problem during backpropagation through time (BPTT). When sequences are long, gradients can shrink exponentially (vanish) or grow uncontrollably (explode), making training unstable and preventing the model from learning long-term dependencies effectively. Although LSTMs were specifically designed to mitigate the vanishing gradient issue using gating mechanisms, they are not completely immune, especially for extremely long sequences.
2. **Difficulty in Modeling Very Long-Term Dependencies:** While LSTMs improve upon standard RNNs by introducing memory cells and gates, they still struggle when dependencies span very long sequences. In practical NLP tasks with lengthy documents, important contextual information may still be diluted over time. More advanced architectures such as Transformers often outperform RNN/LSTM models in capturing long-range relationships.
3. **High Computational Time Due to Sequential Processing:** RNNs and LSTMs process data sequentially, meaning each time step depends on the previous one. This limits parallelization during training and increases computation time compared to feedforward networks or Transformer-based models. For large datasets and long sequences, training becomes slower and less efficient, even with GPU acceleration.
4. **Large Number of Parameters in LSTM:** LSTMs contain multiple gates (input, forget, output) with separate weight matrices, significantly increasing the number of parameters compared to simple RNNs. This increases memory usage and computational cost. Larger models also increase the risk of overfitting when the training dataset is not sufficiently large.

Workflow:

1. **Dataset Loading:** The built-in Keras IMDB movie review dataset is loaded using TensorFlow. It contains 25,000 training reviews and 25,000 testing reviews labeled as positive or negative. Each review is preprocessed and represented as a sequence of integer word indices based on word frequency ranking.
2. **Data Configuration and Vocabulary Limitation:** Only the top 10,000 most frequent words are considered (MAX_FEATURES = 10000). This reduces vocabulary size, lowers computational cost, and removes rare words that contribute little to model learning.
3. **Sequence Padding:** Since reviews have variable lengths, all sequences are padded (or truncated) to a fixed length of 200 words (MAX_LEN = 200) using pad_sequences(). This ensures uniform input shape required for batch processing in the LSTM model.

4. **Model Architecture Definition (Embedding + LSTM):** A Sequential deep learning model is defined consisting of Embedding Layer, LSTM Layer, Dropout Layer, Dense Output Layer.
5. **Hyperparameter Tuning (Keras Tuner – Random Search):** The RandomSearch tuner explores different hyperparameter combinations. The objective is to maximize validation accuracy. Multiple trials are executed to find the best-performing configuration.
6. **Initial Search Training Phase:** Each hyperparameter configuration is trained for a small number of epochs (e.g., 3 epochs) with a validation split of 20% to quickly evaluate performance and identify optimal parameters.
7. **Best Model Reconstruction:** After tuning, the best hyperparameters are selected, and the model is rebuilt using those optimal values to ensure improved generalization performance.
8. **Final Model Training:** The optimized model is trained for extended epochs (e.g., 7 epochs) using 80% of the training data while reserving 20% for validation. This phase allows the model to fully learn sentiment patterns from text sequences.
9. **Model Evaluation:** The trained LSTM model is evaluated on the unseen test dataset to compute final test accuracy and loss. This measures the model's ability to generalize to new reviews.



Performance Analysis:

The performance of the hyperparameter-tuned LSTM-based RNN is evaluated using validation accuracy during tuning, training vs validation learning curves, and final test set performance. Since the IMDB dataset is a supervised binary text classification problem, evaluation focuses on sentiment prediction accuracy and generalization capability.

1. **Test Set Performance:** Final evaluation on unseen test data:
    a. Test Accuracy: 84.61%
    b. Test Loss: 0.4842

    The test accuracy is slightly lower than peak validation accuracy (87.32%), which is expected due to generalization differences. An accuracy of ~84–85% is considered strong performance for a simple single-layer LSTM without advanced techniques.
2. **Generalization Analysis:**
    a. Training Accuracy: Very high (97.8%)
    b. Validation Accuracy: ~87%
    c. Test Accuracy: ~84.6%

    The noticeable gap between training and test accuracy suggests:
    a. The model has high capacity

b. Slight overfitting is present
c. Regularization (dropout 0.3) helped but did not fully eliminate overfitting

Increasing dropout, reducing LSTM units, or using early stopping may improve generalization.

3. **Confusion Matrix Interpretation:**
   Based on the confusion matrix, the majority of reviews are correctly classified, with relatively fewer false positives and false negatives.
   a. The majority of positive and negative reviews are correctly classified.
   b. Some misclassification likely occurs in:
      i. Reviews with mixed sentiment
      ii. Sarcasm
      iii. Neutral or ambiguous wording

   Text sentiment is inherently subjective, so perfect classification is unrealistic.

Hyperparameter Tuning:

LSTM-based Recurrent Neural Networks are highly sensitive to hyperparameter choices, as they directly influence memory capacity, learning stability, convergence speed, and generalization performance. Manual tuning is inefficient due to the large combination space of embedding size, hidden units, dropout, and learning rate. Therefore, automated hyperparameter tuning is performed using Keras Tuner with the Random Search algorithm. The tuner evaluates multiple LSTM configurations and selects the best-performing model based on validation accuracy. The tuned hyperparameters in this experiment are:

1. **Embedding Dimension:** The embedding layer converts integer word indices into dense vector representations. The embedding dimension is tuned between 32–128 in steps of 32. Smaller dimensions reduce computational cost and overfitting. Larger dimensions capture richer semantic relationships between words but increase model complexity. The optimal configuration selects an embedding dimension of 32, indicating that a compact word representation is sufficient for sentiment classification on the IMDB dataset.
2. **Number of LSTM Units:** The LSTM layer's hidden units determine the model's ability to capture sequential dependencies and contextual relationships within text. The search range is 32–128 units in steps of 32. Fewer units may underfit and fail to capture complex sentiment patterns. More units increase learning capacity but raise overfitting risk and computational cost. The best configuration selects 64 LSTM units, providing a balanced trade-off between learning expressive sequence representations and maintaining generalization.
3. **Dropout Regularization Rate:** Dropout is tuned between 0.2–0.5 in steps of 0.1 to reduce overfitting by randomly disabling neurons during training. Lower dropout may not sufficiently prevent overfitting. Higher dropout may reduce learning efficiency. The tuner selects a 0.3 dropout rate, indicating moderate regularization to control the high capacity of the LSTM layer while preserving useful information flow.
4. **Learning Rate:** Tuned among 0.01, 0.001, and 0.0001. The optimal value selected was 0.001, ensuring stable convergence.

Code & Output:

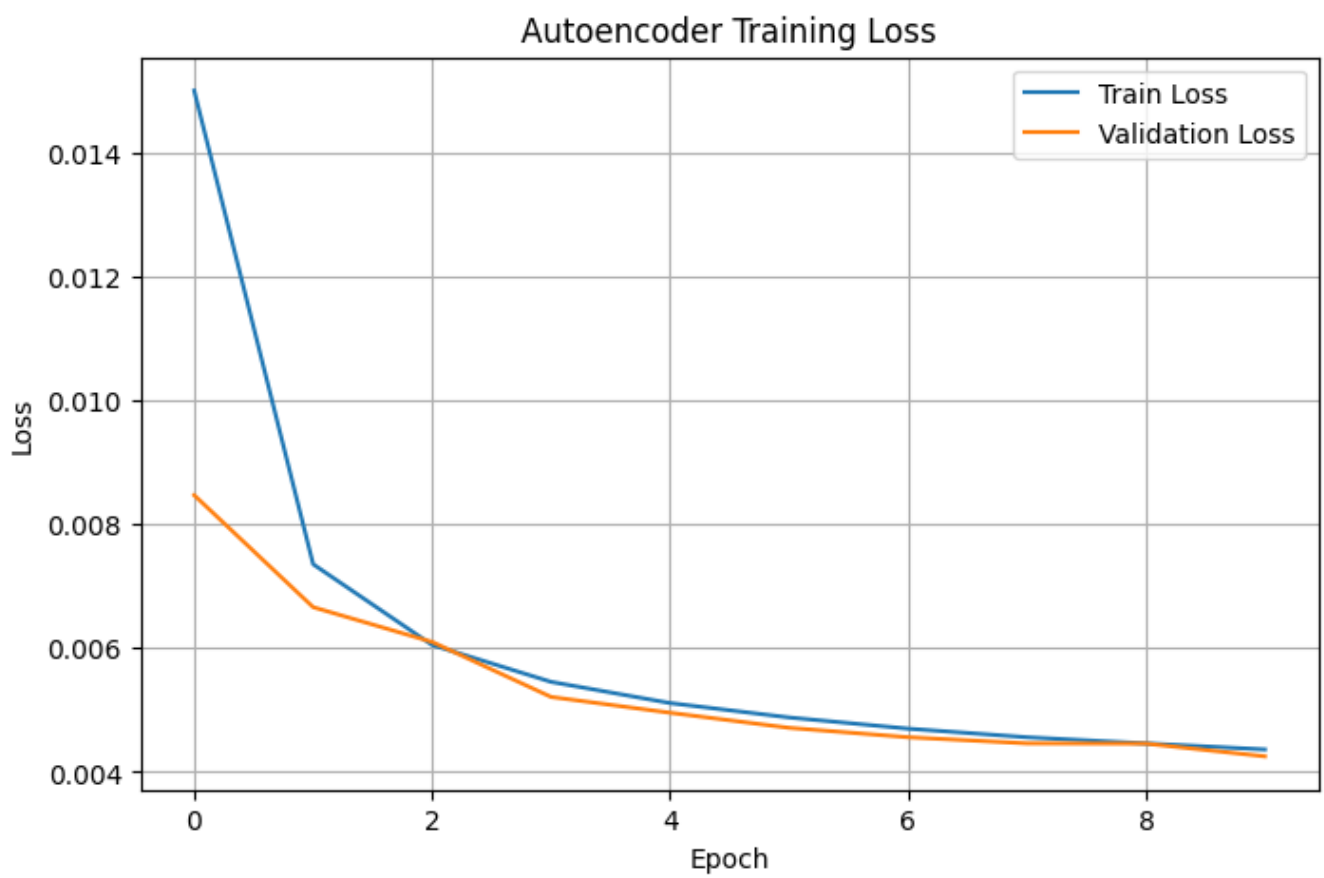
```
# ==========================================
# STEP 1: Import Libraries
# ==========================================
import tensorflow as tf
from tensorflow import keras
import matplotlib.pyplot as plt
import numpy as np

print("TensorFlow Version:", tf.__version__)

# ==========================================
# STEP 2: Load Dataset
```

```
# ==========================================
(x_train, _), (x_test, _) = keras.datasets.cifar10.load_data()

# Normalize
x_train = x_train / 255.0
x_test = x_test / 255.0

print("\nDataset Shape:", x_train.shape)

# ==========================================
# STEP 3: Build CNN Autoencoder (IMPROVED)
# ==========================================
input_img = keras.Input(shape=(32, 32, 3))

# -------- Encoder --------
x = keras.layers.Conv2D(32, (3,3), activation='relu',
padding='same')(input_img)
x = keras.layers.MaxPooling2D((2,2), padding='same')(x)

x = keras.layers.Conv2D(16, (3,3), activation='relu',
padding='same')(x)
encoded = keras.layers.MaxPooling2D((2,2), padding='same')(x)

# -------- Decoder --------
x = keras.layers.Conv2D(16, (3,3), activation='relu',
padding='same')(encoded)
x = keras.layers.UpSampling2D((2,2))(x)

x = keras.layers.Conv2D(32, (3,3), activation='relu',
padding='same')(x)
x = keras.layers.UpSampling2D((2,2))(x)

decoded = keras.layers.Conv2D(3, (3,3), activation='sigmoid',
padding='same')(x)

# Model
autoencoder = keras.Model(input_img, decoded)

autoencoder.compile(optimizer='adam', loss='mse')

autoencoder.summary()

# ==========================================
# STEP 4: Train Model
# ==========================================
history = autoencoder.fit(
   x_train, x_train,
   epochs=10,
   batch_size=128,
   validation_data=(x_test, x_test)
)

# ==========================================
# STEP 5: Plot Loss Graph (MATCH STYLE)
# ==========================================
plt.figure(figsize=(8,5))
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Autoencoder Training Loss')
plt.legend()
plt.grid(True)
plt.show()
```

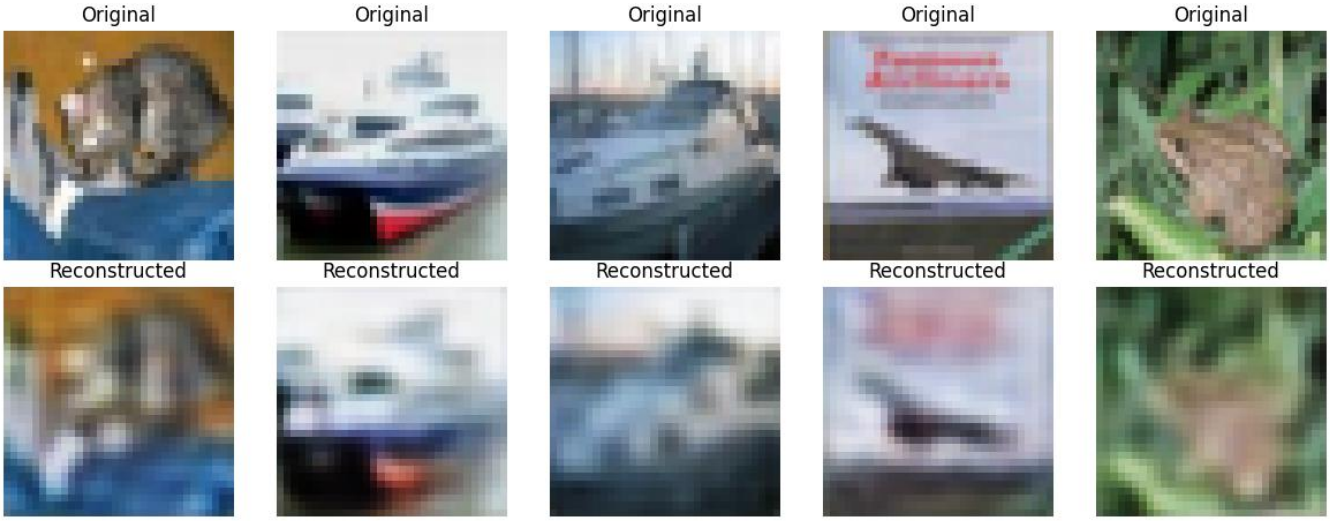
```
# ==========================================
# STEP 6: Reconstruction
# ==========================================
decoded_imgs = autoencoder.predict(x_test)

# ==========================================
# STEP 7: Show Original vs Reconstructed (CLEAN VIEW)
# ==========================================
n = 5
plt.figure(figsize=(12,5))

for i in range(n):
   # Original
   plt.subplot(2, n, i + 1)
   plt.imshow(x_test[i])
   plt.title("Original")
   plt.axis('off')

   # Reconstructed
   plt.subplot(2, n, i + 1 + n)
   plt.imshow(decoded_imgs[i])
   plt.title("Reconstructed")
   plt.axis('off')

plt.suptitle("Original vs Reconstructed Images", fontsize=14)
plt.tight_layout()
plt.show()

# ==========================================
# STEP 8: Random Image Reconstruction
# ==========================================
idx = np.random.randint(0, len(x_test))

original = x_test[idx]
reconstructed = autoencoder.predict(original.reshape(1,32,32,3))

plt.figure(figsize=(8,4))

plt.subplot(1,2,1)
plt.imshow(original)
plt.title("Original Image")
plt.axis('off')

plt.subplot(1,2,2)
plt.imshow(reconstructed[0])
plt.title("Reconstructed Image")
plt.axis('off')

plt.suptitle("Random Sample Reconstruction", fontsize=14)
plt.show()

# ==========================================
# STEP 9: Multiple Random Samples (LIKE YOUR FILE STYLE)
# ==========================================
plt.figure(figsize=(12,6))

for i in range(10):
   idx = np.random.randint(0, len(x_test))

   # Original
   plt.subplot(2,10,i+1)
   plt.imshow(x_test[idx])
   plt.axis('off')
```

```
    # Reconstructed
    plt.subplot(2,10,i+11)
    plt.imshow(decoded_imgs[idx])
    plt.axis('off')

plt.suptitle("Multiple Reconstructions", fontsize=14)
plt.show()
```

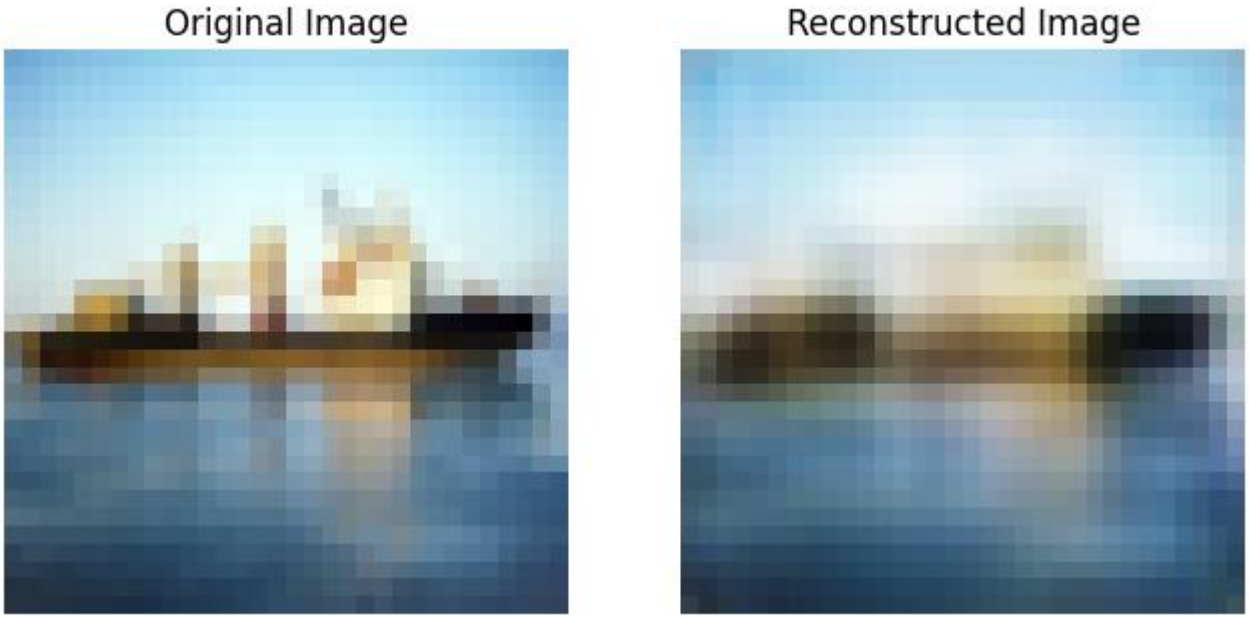
Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 32, 32, 3)	0
conv2d (Conv2D)	(None, 32, 32, 32)	896
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_1 (Conv2D)	(None, 16, 16, 16)	4,624
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 16)	0
conv2d_2 (Conv2D)	(None, 8, 8, 16)	2,320
up_sampling2d (UpSampling2D)	(None, 16, 16, 16)	0
conv2d_3 (Conv2D)	(None, 16, 16, 32)	4,640
up_sampling2d_1 (UpSampling2D)	(None, 32, 32, 32)	0
conv2d_4 (Conv2D)	(None, 32, 32, 3)	867



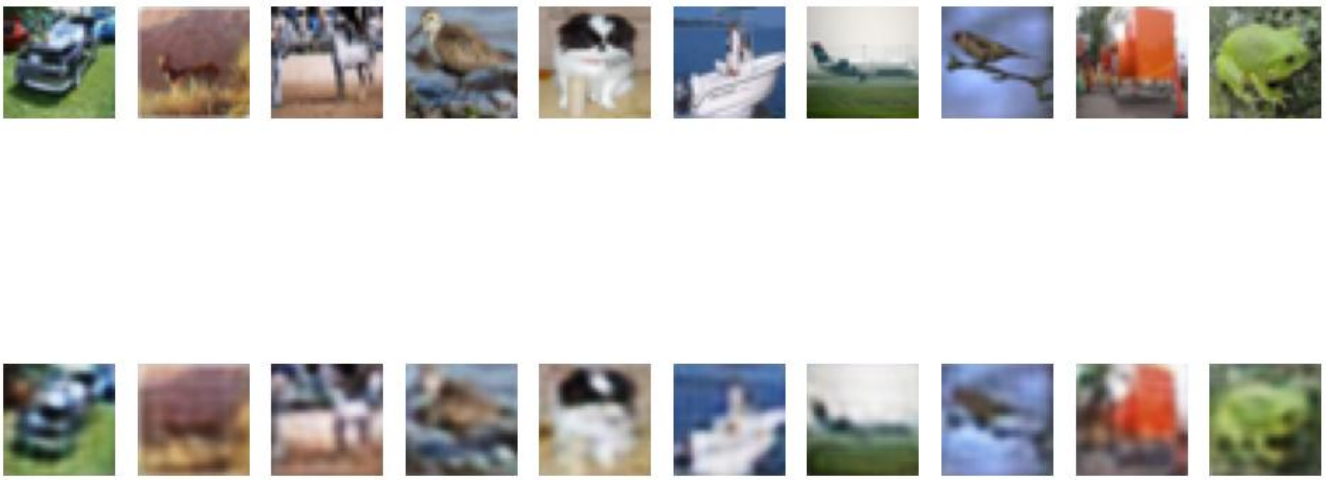
Original vs Reconstructed Images



Random Sample Reconstruction



Multiple Reconstructions



Conclusion:

The experiment successfully implemented and tuned an LSTM-based Recurrent Neural Network for binary sentiment classification using the IMDB dataset. The hyperparameter-optimized model achieved a strong test accuracy of 84.61%, demonstrating effective learning of contextual patterns in text data. Although slight overfitting was observed, the model showed good generalization performance. Overall, the experiment confirms that LSTM networks are well-suited for sequential text modeling and sentiment analysis tasks.